

mEm Inc

3ND EDITION

Lua

Programming

**The Ultimate Beginner's Guide to
Learn Lua Step by Step**

CLAUDIA ALVES & ALEXANDER ARONOWITZ



Lua Programming

The Ultimate Beginner's Guide to Learn Lua Step by Step

Third edition

2021

By
Claudia Alves
& Alexander Aronowitz

TABLE OF CONTENTS

Introduction

The audience

About the third edition

Other resources

Certain Typographical Conventions

Running examples

Acknowledgments

PART I

Language

Chapter 1. Getting Started

1.1. Blocks

1.2. Some lexical conventions

1.3. Global variables

1.4. Separate interpreter

Exercises

Chapter 2. Types and values

2.1. Nil

2.2. Boolean

2.3. Numbers

2.4. Strings

Literals

Long lines

Type conversions

2.5. Tables

2.6. Functions

2.7. userdata and threads

Exercises

Chapter 3. Expressions

3.1. Arithmetic Operators

3.2. Comparison operators

3.3. Logical operators

3.4. Concatenation

3.5. Length operator

3.6. Operator Priorities

3.7. Table constructors

Exercises

Chapter 4. Operators

4.1. Assignment operators

4.2. Local variables and blocks

4.3. Control structures

if then else

while

repeat

Numeric for statement

General for statement

4.4. break, return and goto

Exercises

Chapter 5. Functions

5.1. Multiple Results

5.2. Functions with a variable number of arguments

5.3. Named arguments

Exercises

Chapter 6. More about functions

6.1. Closures

6.2. Non-global functions

6.3. Optimization of tail calls

Exercises

Chapter 7. Iterators and the generic for

7.1. Iterators and Closures

7.2. Generic for semantics

7.3. Stateless iterators

7.4. Complex State Iterators

7.5. True iterators

Exercises

Chapter 8. Compilation, Execution, and Errors

8.1. Compilation

8.2. Precompiled Code

8.3. C code

8.4. Errors

8.5. Error and Exception Handling

8.6. Error messages and call stack

Exercises

Chapter 9. Coroutines

9.1. Basics of coroutines

9.2. Channels and Filters

9.3. Coroutines as iterators

9.4. Non-displacing multi-threading
Exercises

Chapter 10. Completed Examples

10.1. The problem of eight queens

10.2. The most common words

10.3. Markov chain

Exercises

PART II

Tables and Objects

Chapter 11. Data Structures

11.1. Arrays

11.2. Matrices and multidimensional arrays

11.3. Linked Lists

11.4. Queues and double queues

11.5. Sets and Sets

11.6. Line buffers

11.7. Counts

Exercises

Chapter 12. Data Files and Persistence

12.1. Data files

12.2. Serialization

Saving tables without loops

Saving tables with loops

Exercises

Chapter 13. Metatables and Metamethods

13.1. Arithmetic metamethods

13.2. Comparison Methods

13.3. Library Metamethods

13.4. Methods for accessing the table

__Index metamethod

`__Newindex` metamethod

Default tables

Tracking access to a table

Read-only tables

Exercises

Chapter 14. Environment

14.1. Global variables with dynamic names

14.2. Descriptions of global variables

14.3. Non-global environments

14.4. Using `_ENV`

14.5. `_ENV` and `load`

Exercises

Chapter 15. Modules and Packages

15.1. `Require` function

Renaming a module

Search along the path

File crawlers

15.2. The Standard Approach for Writing Lua Modules

15.3. Using environments

15.4. Submodules and Packages

Exercises

Chapter 16. Object Oriented programming

16.1. Classes

16.2. Inheritance

16.3. Multiple inheritance

16.4. Hiding

16.5. Single Method Approach

Exercises

Chapter 17. Weak Tables and Finalizers

17.1. Weak tables

- 17.2. Functions with caching
- 17.3. Object Attributes
- 17.4. Again tables with default values
- 17.5. Ephemeral tables
- 17.6. Finalizers

Exercises

PART III

Standard Libraries

Chapter 18. Math Library

Exercises

Chapter 19. Library for Bitwise Operations

Exercises

Chapter 20. Library for Working with Tables

20.1. Insert and remove functions

20.2. Sorting

20.3. Concatenation

Exercises

Chapter 21. Library for working with strings

21.1. Basic functions for working with strings

21.2. Functions for working with templates

String.find function

String.match function

String.gsub function

String.gmatch function

21.3. Templates

21.4. Grips

21.5. Substitutions

URL encoding

Replacing tabs

21.6. Tricky tricks

21.7. Unicode

Exercises

Chapter 22. Library I / O

22.1. Simple I / O Model

22.2. Full I / O model

A small trick to increase performance

Binaries

22.3. Other operations with files

Exercises

Chapter 23. Library of operating room functions systems

23.1. Date and time

23.2. Other system calls

Exercises

Chapter 24. Debug Library

24.1. Accessibility (introspection)

Accessing Local Variables

Access to non-local variables

Access to other coroutines

24.2. Hooks

24.3. Profiling

Exercises

PART IV

WITH API

Chapter 25. Overview of the C API

25.1. First example

25.2. Stack

Putting items on the stack

Referring to items

Other stack operations

25.3. Error handling in the C API

Handling errors in the application code

Error handling in the library code

Exercises

Chapter 26. Extending Your Application

26.1. Basics

26.2. Working with tables

26.3. Lua function calls

26.4. Generalized function call

Exercises

Chapter 27. Calling C from Lua

27.1. Functions on C

27.2. Continuations

27.3. C modules

Exercises

Chapter 28. Techniques for writing functions in C

28.1. Working with arrays

28.2. Working with strings

28.3. Saving state in functions on C

Register

Function related values

Function related values used

several functions

Exercises

Chapter 29. User-Defined Types in C

29.1. User data (userdata)

29.2. Metatables

29.3. Object Oriented Access

29.4. Access as an ordinary array

29.5. Light objects of type userdata (light userdata)

Exercises

Chapter 30. Resource Management

30.1. Directory iterator

30.2. XML Parser

Exercises

Chapter 31. Threads and States

31.1. Numerous threads

31.2. Lua States

Exercises

Chapter 32. Memory Management

32.1. Memory allocation function

32.2. Garbage collector

Garbage collector API

Exercises

INTRODUCTION

When Waldemar, Louis, and I started developing Lua in 1993, we could hardly imagine that Lua would spread like that. On- started as home language for two specific projects, now Lua is widely used in all areas one can get

benefits from simple, extensible, portable and efficient scripting language such as embedded systems, mobile devices swarms and, of course, games.

We designed Lua from the beginning to integrate with software software written in C / C ++ and other common strange languages. There are many benefits to this integration. Lua is a tiny and simple language, partly because it doesn't tries to do what C is already good at, such as speed, low-level operations and interaction with third-party programs parties. Lua relies on C for these tasks. Lua offers something for which C is not good enough: sufficient distance from the hardware go support, dynamic structures, no redundancy and ease of testing and debugging. For these

purposes, Lua has safe environment, automatic memory management and good possibilities for working with strings and other types resizable data. Some of Lua's strength comes from its libraries. And this is no coincidence. In the end Finally, one of the main strengths of Lua is extensibility. Many language features contribute to this. Dynamic typing provides a large degree of polymorphism. Automatic memory management simplifies interfaces because there is no need to the ability to decide who is responsible for allocating and freeing memory or how to handle overflows. Higher-order functions and anonymous functions allow a high degree of parameterization, making functions more versatile. More than an extensible language, Lua is a “ *glue vayushim* ”(*glue*) *language* . Lua supports a component-based approach to software development when we create an application by gluing together existing high-level components. These components are written in a compiled language with a static typing such as C / C ++; Lua is the glue we use use to arrange and connect these components. Usually components (or objects) represent more specific low- tier entities (such as widgets and data structures), which which hardly change during the development of the program and which take up the bulk of the final program execution time. Lua gives the final form to the application, which is most likely changes a lot during the life of a given software product. However, unlike other "glue" technologies, Lua is is a complete programming language. Therefore we can use Lua not only to "glue" components, but also to adaptation and customization of these components, as well as to create a floor new components. Of course, Lua isn't the only scripting language. Exists other languages you can use for roughly the same goals. Nevertheless, Lua provides a whole set of possibilities, which make him the best choice for many tasks and gives him your unique profile:

- *Extensibility* . Lua's extensibility is so great that many consider Lua not as a language, but as a set for DSL structures (domain-specific language, a language created for specific area, application). We developed Lua from the beginning so that it is extensible both through code on Lua as well as through C code. How the Lua proof implementsmost of its basic functionality via external library. Interaction with C / C ++ really simple and Lua has been successfully integrated with many others languages such

as Fortran, Java, Smalltalk, Ada, C #, and even with scripting languages like Perl and Python.

- *Simplicity* . Lua is a simple and small language. It is founded on a small number of concepts. This simplicity makes it easier to study nie. Lua contributes to its very small size. Floor- distribution kit (source code, manual, binaries ly for some platforms) is quietly placed on one floppy disk.
- *Efficiency* . Lua has a very efficient implementation tions. Independent tests show that Lua is one of the most most fast languages among scripting languages.
- *Portability* . When we talk about portability, we say Rome on running Lua on all platforms you are only talking about heard: all versions of Unix and Windows, PlayStation, Xbox, Mac OS X and iOS, Android, Kindle Fire, NOOK, Haiku, QUALCOMM Brew, big servers from IBM, RISC OS, Symbian OS, process Sora Rabbit, Raspberry Pi, Arduino and more. Original the code for each of these platforms is almost the same. Lua does not use conditional compilation to adapt its code for different machines, instead it keeps standard ANSI (ISO) C. Thus, you usually do not you need to adapt it to the new environment: if you have a com- piler with ANSI C, then you just need to compile Lua.

The audience

Lua users generally fall into one of three broad groups: those who use Lua already built into the application, those who use use Lua separately from any application (standalone), and those who use Lua and C together. Many people use Lua built into some application, like Adobe Lightroom, Nmap, or World of Warcraft. These pri- applications use the Lua-C API to register new functions, creating new types and changing the behavior of some operations language by configuring Lua for its area. Often users are whom applications don't even know that Lua is an independent language, adapted for the given area. For example, many developers Lightroom plugins are unaware of other uses of this language; Nmap users generally view Lua as

scripting Nmap language; World of Warcraft players can view Lua as a language exclusively for this game.

Lua is also useful as just an independent language, not only for word processing and one-time small programs, but also for a variety of medium to large sized projects. For the In this way, the main functionality of Lua comes from its libraries. Standard libraries, for example, provide basic new functionality for working with templates and other functions for working with strings. As Lua improves its support libraries, a large number of external packages appeared. Lua Rocks, a system for building and managing modules for Lua, now has over 150 packages. Finally, there are programmers who use Lua as a library flow for C. Such people write more in C than in Lua, although they need requires a good understanding of Lua to create interfaces that are simple, easy to use and well integrated tied with the tongue.

This book can be useful to all of these people. The first part covers the tongue itself, showing how the whole its potential. We focus on various language constructs and use numerous examples and exercises to show know how to use them for practical tasks. Some chap- this part covers basic concepts such as managing structures, while the rest of the chapters cover more advanced tricky topics like iterators and coroutines.

The second part is completely devoted to tables, the only structure data tour in Lua. The chapters in this part discuss data structures, persistence, packages and object-oriented programming. It is there that we will show the full power of language. The third part introduces the standard libraries. This part especially useful for those who use Lua on their own language, although many applications include part or all standard libraries. In this part, each library is dedicated separate chapter: math library, bitwise library, library for working with tables, library for working with strings mi, I / O library, operating system library and debug library. Finally, the last part of the book covers the API between Lua and C.

This section is markedly different from the rest of the book. In this part we will be programming in C, not Lua. For some, this part may be uninteresting, but for someone - on the contrary, the most useful part of the book.

Running examples

You will need a Lua interpreter to run the examples from this books. Ideally, you should use Lua 5.2, however most in the examples will work on Lua 5.1 without any changes. The Lua site (<http://www.lua.org>) stores all the source code for interpreter. If you have a C compiler and know how compile the C code on your computer, you better ask try to install Lua from source; it's really easy. The *Lua Binaries* site (look for luabinaries) already offers compiled native interpreters for all major platforms. If you using Linux or another UNIX-like system, you can check the repository of your distribution; many distributions already offer ready-made packages with Lua. For Windows, a good choice is rum is *Lua for Windows* (look for luaforwindows), which is a convenient set for working with Lua. It includes interpreting torus, integrated editor and many libraries.

If you are using Lua embedded in an application like WoW or Nmap, then you may need a manual for this. placement (or the help of a "local guru") in order to understand, how to run your programs. Nevertheless, Lua remains all the same the same language; most of the examples we will look at in this book are applicable regardless of how you use Lua. But I recommend that you start learning Lua with an interpreter to run you examples.

Part I

I am the language

CHAPTER 1

Begin

Continuing the tradition, our first Lua program simply printed em “Hello World” :

`print (“Hello World”)` If you are using a separate Lua interpreter, then all you need is you need to run your first program - this is to run the interpreter tator - usually called `lua` or `lua5.2` - with the name of the text file containing your program. If you saved the above the above program in `hello.lua` file , then you should run following command:

```
% lua hello.lua
```

As a more complex example, our next program defines There is a function for calculating the factorial of a given number, asking gives the user a number and prints its factorial:

- defines a factorial function

```
function fact (n)
if n == 0 then
return 1
else
return n * fact (n-1)
end
end
print (“enter a number:”)
a = io.read (“* n”) - reads a number
print (fact (a))
```

1.1. Blocks

Every piece of code that Lua executes, such as a file or from- a smart string in interactive mode is called *a* chunk. A block is simply a sequence of commands (or statements). Lua does not need a separator between consecutive operators, but you can use semicolon if you like. I personally use I use a semicolon only to separate statements written in one line. Line

splitting does not play any role in the syntax system Lua; so, the following four blocks are valid and equivalent:

```
a = 1
b = a * 2
a = 1;
b = a * 2;
a = 1; b = a * 2
a = 1 b = a * 2 - ugly, but valid
```

A block can consist of just one statement, as in the example "Hello World", or consist of a set of operators and definitions functions (which are actually just assignments, as we will see later), as in the factorial example. Block can be as great as you want. Since Lua is also used as language for describing data, blocks of several megabytes are not a rarity. The Lua interpreter does not have any problems with both with large blocks. Instead of writing your programs to a file, you can run the interpreter interactively. If you run those lua without arguments, then you will see its prompt for input:

```
% lua
Lua 5.2 Copyright (C) 1994-2012 Lua.org, PUC-Rio
>
```

Accordingly, each command that you enter (like, for example measure, print "Hello World") is executed immediately after how you enter it. To exit the interpreter, just type end-of-file character (ctrl-D in UNI, ctrl-Z in Windows) or call the exit function from the operating system library - you need type `os.exit()` .

In interactive mode, Lua usually interprets each the line you enter as a separate block. However, if he is detects that the line is not a complete block, then it waits continue typing until a complete block is obtained. This way you can enter multi-line definitions, so as a factorial function , directly interactively. One- but it is usually more convenient to place such definitions to a file and then call Lua to execute that file. You can use the `-i` option to force Lua switch to interactive mode after executing the given block ka:

```
% lua -i prog
```

A command like this will execute the block in the prog file and then go into interactive mode. This is especially useful for debugging and manual testing. At the end of this chapter, we will look at other options. command line for the Lua interpreter. Another way to trigger blocks is the dofile function, which immediately executes the file. For example, let's say you have lib1.lua file with the following code:

```
function norm (x, y)
return (x ^ 2 + y ^ 2) ^ 0.5
end
function twice (x)
return 2 * x
end
```

Then interactively you can type

```
> dofile ("lib1.lua") - load your library
> n = norm (3.4, 1.0)
> print (twice (n)) -> 7.0880180586677
```

The dofile function is also useful when you are testing a piece of code. You can work with two windows: one contains the text editor with your program (for example, in the prog.lua file), and in other window is a console running the Lua interpreter in the interactive mode. After you have saved the changes to your program, you do dofile ("prog.lua") in the console to load the new code; then you can start using the new code, calling functions and printing the results.

1.2. Some lexical agreements

Identifiers (or names) in Lua are strings from Latin their letters, numbers and underscores that do not start with a number;

eg:

```
ij i10 _ij
aSomewhatLongName _INPUT
```

You are better off avoiding identifiers consisting of underscores followed by capital Latin letters (for example, _VERSION); they are

reserved for special purposes in Lua. I usually use id _ (single underscore) for dummy variables. In older versions of Lua, the concept of what a letter is depended on from the locale. However, these letters make your program unsuitable to run on systems that do not support this locale. Therefore, Lua 5.2 considers only letters as letters. from the following ranges: AZ and az . The following words are reserved, you cannot use them as identifiers:

```
and break do
else elseif
end false goto for function
if in
local nil not
or repeat return then true
until while
```

Lua is case sensitive: **and** **is** a reserved word, however, And and AND are two different identifiers. The comment starts with two minus signs (-) and continues with- until the end of the line. Lua also supports block comments, which starts with - [[and goes to the next]] . Standard the way to comment out a piece of code is to put it between

```
- [[ and -]] as shown below:
- [[
print (10) - no action (commented out)
-]]
```

To make this code active again, just add one minus to the first line:

```
--- [[
print (10) -> 10
-]]
```

In the first example - [[in the first line starts a block commentary, and the double minus in the last line is also in this comment. In the second example --- [starts the usual single line comment, so the first and last lines become regular independent comments. In this case print is outside of comments.

1.3. Global Variables

Global variables do not need descriptions; you just use them you whine. It is not an error to refer to uninitialized variable; you just get nil as a result-that:

```
print (b) -> nil
b = 10
print (b) -> 10
```

If you assign nil to a global variable, Lua will itself as if this variable has never been used:

```
b = nil
print (b) -> nil
```

After this assignment, Lua may eventually regain its password. the space occupied by this variable.

1.4. Separate interpreter

Stand-alone interpreter (also called lua.c in relation to the name of its source file or just lua by name executable file) is a small program that allows direct use of Lua. This section presents her basic options.

When the interpreter loads a file, it skips the first string if it starts with '#' character . This allows you to use make Lua a scripting interpreter on UNIX systems. If you start your script with something like

```
#!/usr/local/bin/lua
```

(assuming the interpreter is in /usr/local/bin) or

```
#!/usr/bin/env lua ,
```

then you can directly run your script without explicitly start the Lua interpreter. The interpreter is called like this:

```
lua [options] [script [args]]
```

All parameters are optional. As we have seen, when we start If lua has no arguments, it goes into interactive mode.

The -e option allows you to directly specify the code directly in the command- line like below:

```
% lua -e "print (math.sin (12))" -> -0.53657291800043
```

(UNIX requires double quotes so that the command interpreter the torus (shell) did not parse parentheses).

The -l option loads the library. As we have seen previously, -i pe- puts the interpreter into interactive mode after processing the OS arguments. So the next call will load the library lioteku lib , then execute the assignment x = 10 and finally pass into interactive mode.

```
% lua -i -llib -e "x = 10"
```

Interactively, you can print the value of the expression by simply typing a line that starts with an equal sign for followed by the expression:

```
> = math.sin (3) -> 0.14112000805987  
> a = 30  
> = a -> 30
```

This feature allows Lua to be used as a calculator. Before executing its arguments, the interpreter looks for re- environment variable named LUA_INIT_5_2 or, if such a change no, no, LUA_INIT . If one of these variables is present and its has the form @filename , the interpreter launches this file. If LUA_INIT_5_2 (or LUA_INIT) is defined but not start- is started with the '@' character , then the interpreter assumes that it contains executable Lua code and executes it. LUA_INIT gives huge the ability to configure the interpreter, since when configuration, all the power of Lua is available to us. We can download packages, change the current path, define your own functions change, rename or delete functions, etc. The script can get its arguments in a global variable

arg . If we have a call like % lua script abc , then the interpreter creates arg table with all command line arguments before by executing the script. The script name is located at index 0, per- the first argument (in the example it is "a") is located at index 1, and so on. The preceding options are arranged in negative indices, as they appear before the script name. For example, ra- look at the following call:

```
% lua -e "sin = math.sin" script ab
```

The interpreter collects arguments as follows:

```
arg [-3] = "lua"  
arg [-2] = "-e"  
arg [-1] = "sin = math.sin"  
arg [0] = "script"  
arg [1] = "a"  
arg [2] = "b"
```

Most often the script uses only positive indices (in the example, these are `arg [1]` and `arg [2]`).

Since Lua 5.1 the script can also receive its arguments using the expression `...` (three dots). In the main part of the script, this expression gives all the arguments of the script (we will discuss similar expressions (see section 5.2)).

Exercises

Exercise 1.1 . Run the factorial example. What will happen with your program if you enter a negative number? Modify the example to avoid this problem.

Exercise 1.2 . Run the twice example by loading once file using the `-l` option , and another time using `dofile` . what faster?

Exercise 1.3 . Can you name another language using `--` for comments?

Exercise 1.4 . Which of the following lines are valid by our identifiers?
____ _end End end until? nil NULL

Exercise 1.5 . Write a simple script that prints your name without knowing it in advance.